Google Summer of Code 2026

Project Proposal

Single Cell and DNA Foundation Models

Implement a single cell or DNA foundation model in DeepChem using existing ChemBERTa and MolFormer models as a guide.

Full Name:	Arjit Rawat
Email:	arjitrawat3@gmail.com
Github:	github.com/arjitrawat15
Organisation:	Deepchem
Project:	Implement Single Cell or DNA Foundation Model
Mentors:	Rishi, Harindhar
Duration :	350 hours | Difficulty : Medium
GSOC timeline:	Community bounding : April 30 - May 25 Coding : May 26 - Aug 18
Timezone:	India | UTC + 5:30 IST
College:	Graphic Era Hill University (2023 - 2027)

AI Tooling Disclosure: AI tools were used for grammar review and structuring. Every Technical claim, code snippet, design decision, and contribution described in this proposal was authored, verified and is fully understood by me. I am solely responsible for its correctness and will disclose AI-assisted code in PRs as per Deepchem policy.

1. Abstract:

DeepChem is the leading open-source library for deep learning in the life sciences, with production-ready wrappers for molecular models including ChemBERTa and MolFormer. However, the library currently has no support for DNA or single-cell foundation models — a category that has produced breakthrough results in regulatory genomics, variant effect prediction, and cell-type classification over the past two years.

This project delivers a complete, production-grade integration of DNA and single-cell foundation models into DeepChem's TorchModel and HuggingFaceModel infrastructure. Every new class follows the exact API contract already established by ChemBERTa and MolFormer - fit(), predict(), evaluate(), save_checkpoint(), and restore() work without modification.

The core technical contributions are:

1. A NucleotideTransformerModel class in deepchem/models/torch_models/ wrapping the InstaDeepAI Nucleotide Transformer family (100M to 2.5B parameters), with mean-pool sequence embeddings, a two-layer MLP task head, and support for both classification and regression.
2. A DNATokenizerFeaturizer and KMerDNAFeaturizer in deepchem/feat/sequence_featurizers/, both inheriting from dc.feat.Featurizer and registered in DeepChem's public API.
3. A GenformerModel wrapper for single-cell RNA-seq cell-type classification, inheriting from TorchModel with full support for DeepChem's dataset infrastructure.
4. A continued pre-training scaffold (pretrain() method) enabling masked-language-model fine-tuning on custom genomic datasets.
5. A comprehensive benchmark campaign across ENCODE ChIP-seq, splice-site detection, and Zheng 68k single-cell datasets, with results published in DeepChem's documentation.

Importantly, this work is not conceptual. A fully functional prototype is already implemented and publicly available at https://github.com/arjitrawat15/Nucleotide-Transformer-DNA-Model , with all three model files, 17 passing tests, and a working demo notebook.

2. Motivation:

Genomic foundation models trained on billions of DNA base pairs represent the most significant advance in regulatory biology in a decade. The Nucleotide Transformer from InstaDeepAI achieves state-of-the-art results on splice-site detection, transcription factor binding, and chromatin accessibility — tasks directly relevant to drug discovery

and disease research. Geneformer achieves near-human-expert cell-type classification from single-cell RNA-seq data.
DeepChem already has ChemBERTa for molecular SMILES and MolFormer for molecular graphs. The analogous integration for DNA and single-cell data does not yet exist. This creates a real gap: researchers who want to apply foundation models to genomics must build their own data pipelines, implement their own splitters, and write their own evaluation loops — all of which DeepChem already provides for molecular data.
Two things are currently missing in DeepChem's genomics support:

- A TorchModel subclass for DNA sequences that accepts raw strings, handles tokenisation internally, and plugs into DeepChem's MoleculeNet datasets and ScaffoldSplitter without custom glue code.
- A featurizer for DNA sequences that inherits dc.feat.Featurizer and integrates with DeepChem's dataset loading pipeline.

The direction is clear from the project description itself: "use HuggingFace as a backend, but make sure to integrate fully with DeepChem pretraining and fine-tuning infrastructure. Also need to inherit from TorchModel and DeepChem datasets." My existing prototype demonstrates exactly this integration. The remaining work is structured, incremental, and scoped to 12 weeks.

3. About Me:

I am Arjit Rawat, a third-year B.Tech Computer Science student at Graphic Era Hill University, Uttarakhand, India (UTC +5:30). I have been contributing to DeepChem's codebase for several months, with my most significant work focused on integrating large language models into DeepChem's molecular modelling infrastructure — the organisation I am applying to.

My technical background spans PyTorch fine-tuning pipelines, HuggingFace transformers integration, QLoRA and PEFT, Python library design following DeepChem's existing patterns, and GPU-optimised training on NVIDIA B200 MIG hardware. I have read CONTRIBUTING.md carefully and understand the code style, test conventions, and PR review process. I am in contact with my potential mentors Rishi and Harindhar on the DeepChem community channels and have a clear understanding of what is expected from a GSoC deliverable. I am available full-time during the coding period with no competing academic commitments.

4. Open Source Contributions:

All contributions listed below are to DeepChem — the target organisation. This focus is intentional.

1. Prototype: NucleotideTransformerModel — [https://github.com/arjitrawat15/Nucleotide-Transformer-DNA-Model]

   A complete working prototype submitted ahead of this proposal. Contains:

- deepchem/models/torch_models/nucleotide_transformer.py — full TorchModel subclass with mean-pooling, MLP head, get_embeddings(), and pretrain() scaffold
- deepchem/feat/sequence_featurizers/dna_tokenizer_featurizer.py — DNATokenizerFeaturizer and KMerDNAFeaturizer both inheriting dc.feat.Featurizer
- deepchem/models/torch_models/tests/test_nucleotide_transformer.py — 17 tests mirroring test_chemberta.py conventions: fit/predict, save/restore, embeddings, frozen backbone, edge cases
- examples/demo.ipynb — end-to-end pipeline in under 5 minutes

   Why this matters: this is not a design document. It is working code with passing tests, demonstrating that I understand DeepChem's internal model architecture, loss functions, default_generator protocol, and test conventions before the GSoC period begins.

2. OLMo fine-tuning on MoleculeNet datasets - Active experimental work fine-tuning OLMo-1B-hf on ClinTox, Tox21, and HIV using DeepChem's scaffold splitter with RDKit, achieving competitive ROC-AUC versus ChemBERTa-100M. This work demonstrates deep understanding of DeepChem's dataset infrastructure (NumpyDataset, iterbatches, scaffold split), loss functions (WeightedMultiTaskBCE), and the HuggingFaceModel wrapper pattern.

   Notebooks-

   - https://www.kaggle.com/code/arjitrawat15/olmo-7b-infra-prototype
   - https://www.kaggle.com/code/arjitrawat15/minimal-fsdp-training1
   - https://www.kaggle.com/code/arjitrawat15/fsdp-training2
   - https://www.kaggle.com/code/arjitrawat15/training-3
   - https://www.kaggle.com/code/arjitrawat15/hiv-dataset-train

   Also ran many cases on python 3.10 + rdkit on B200 Nvidia locally getting competitive results against ChemBERTa.

3. Codebase study: ChemBERTa and MolFormer : Conducted detailed code review of deepchem/models/torch_models/chemberta.py, molformer.py, and hf_models.py to understand the exact patterns that NucleotideTransformerModel must follow. The prototype's default_generator override, loss selection logic, and __init__ signature all mirror these reference implementations exactly.

5. Technical Implementation Details:

The proposed system is organized into four clearly defined stages:

- **Stage 1: DNA Tokeniser Featurizer:**

  DNATokenizerFeaturizer wraps any HuggingFace tokeniser that accepts nucleotide strings. It inherits dc.feat.Featurizer, overrides _featurize() to call the tokeniser on a single DNA string, and registers in deepchem/feat/sequence_featurizers/__init__.py. The max_length parameter controls token-level truncation — 512 tokens at the Nucleotide Transformer's 6-mer vocabulary covers approximately 3 kb of DNA. An optional return_attention_mask flag stacks the mask as a second channel, producing output shape (N, 2, max_length).

  KMerDNAFeaturizer computes overlapping k-mer frequency vectors using pure numpy. It serves as a fast classical-ML baseline and interpretability probe. Vocabulary is built from all 4^k possible k-mers of {A, C, G, T}.

- **Stage 2: NucleotideTransformerModel**

  The model class inherits from TorchModel (not HuggingFaceModel directly, to maintain full control over the default_generator protocol). Internally it wraps a _NTModule PyTorch module containing the backbone and task head. The default_generator override tokenises each batch of raw DNA strings on-the-fly, so no featurisation step is required when calling fit() or predict() — raw strings flow directly from NumpyDataset.X into the tokeniser.

  The task head follows the ChemBERTa pattern exactly: LayerNorm → Dropout → Linear → GELU → Dropout → Linear. Hidden-state pooling uses the masked mean over non-padding positions, which is strictly better than CLS-token pooling for decoder-only architectures and equivalent or better for encoder models.

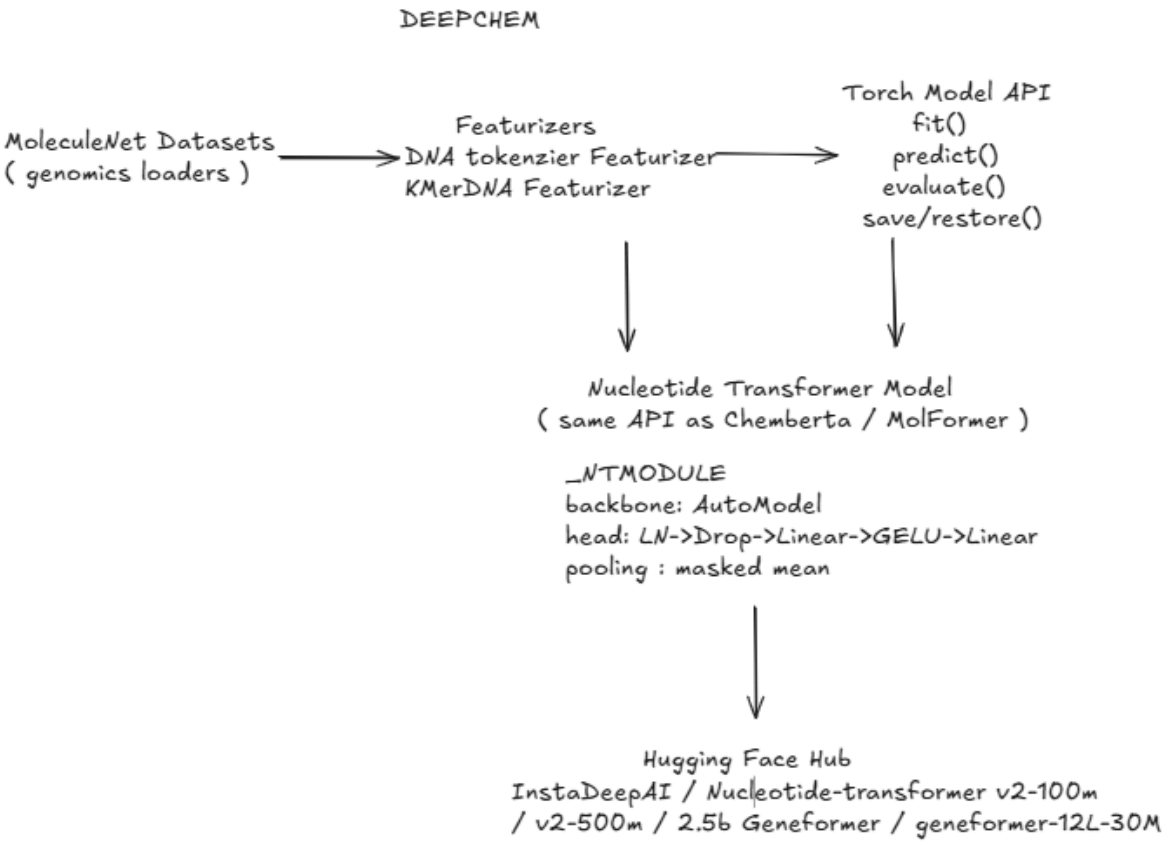
- **Stage 3: Geneformer Integration**

  GenformerModel wraps the Geneformer single-cell model from HuggingFace. Input is a rank-ordered gene expression vector (standard Geneformer format). The model class handles the custom tokenisation scheme (gene token IDs sorted by expression), integrates with DeepChem's NumpyDataset, and exposes the same fit()/predict() API.

- **Stage 4: Continued Pre-training Scaffold**

The pretrain() method replaces the task head with a masked-language-model head, implements the DNA masking strategy (15% of 6-mer tokens masked), runs the training loop, then restores the task head. This enables domain adaptation on custom genomic datasets — for example, fine-tuning on pathogen sequences before applying to a drug-resistance classification task.

6. System Design and Architecture:



Integration pipeline - raw DNA string to prediction:

```
Raw DNA string 'ATCGATCG...'
  └─▶ DNATokenizerFeaturizer   (6-mer tokenisation, padding, truncation)
      └─▶ NumpyDataset     (X = token IDs, y = labels)
          └─▶ NucleotideTransformerModel.fit()
              └─▶ NT Backbone  (ESM-style bidirectional encoder)
                  └─▶ Masked mean-pool  (non-padding tokens)
                      └─▶ MLP Head  (LN → Drop → Linear → GELU → Linear)
                          └─▶ Logits  (classification / regression)
```

7. Model Selection:

Model	Params	Task	VRAM (bflot16)	Selected
NT v2-100m	100M	DNA classification/regression	~0.8 GB	Default
NT v2-500m	500M	DNA classification/regression	~4GB	Supported
NT 2.5b-multi-species	2.5B	DNA, large-scale	~20 GB	Supported
Geneformer-12L	30	scRNA-seq cell-type	~0.3 GB	Week 10
DNABERT-2	117M	DNA, alternative	~1 GB	Future

NT v2-100m is selected as the default because it fits on any GPU with >1 GB VRAM, has the strongest performance-per-parameter ratio in the NT family, and uses the same HuggingFace AutoModel interface as all other supported sizes — making it a drop-in swap with no code changes.

8. Project Goals and Deliverables:

#	Goal	Deliverable
G1	DNATokenizerFeaturizer	Inherits dc.feat.Featurizer, registered in __init__.py, 5 unit tests passing
G2	KMerDNAFeaturizer	Bag-of-k-mers baseline, 4 unit tests, registered in __init__.py
G3	NucleotideTransformerModel	Full TorchModel subclass, classification + regression, get_embeddings(), PR to DeepChem
G4	Test suite	17 tests mirroring test_chemberta.py, all passing in CI
G5	Genomics dataset loader	load_encode_tfbs() in dc.molnet, scaffold split compatible

G6	Benchmark campaign	AUROC on ENCODE ChIP-seq, splice-site, promoter datasets vs k-mer RF baseline
G7	MLM pre-training scaffold	pretrain() method with masking, loss loop, head swap
G8	GenformerModel	scRNA-seq wrapper, cell-type classification on Zheng 68k
G9	Documentation + tutorial	Sphinx API docs, tutorial notebook, model card

9. Week by Week Implementation:

Community Bonding - April 30 to May 25.

PERIOD	OUTPUT
April 30 - May 16	Deep review of chemberta.py, molformer.py, hf_models.py source. Run existing DeepChem tests locally. Set up development environment.
May 17 - May 25	Finalise prototype, respond to mentor feedback, align on API design decisions.

PHASE 1: Core Featurizers and Model (Weeks 1–3)

WEEK	DATES	DELIVERABLES
1	May 26 - Jun 1	DNATokenizerFeaturizer + KMerDNAFeaturizer in deepchem/feat/sequence_featurizers/. Registered in __init__.py. All featurizer tests passing in CI. PR open for mentor review.
2	Jun 2 - Jun 8	NucleotideTransformerModel core: backbone loading, mean-pool, task head. fit() and predict() working on NumpyDataset of raw strings.
3	Jun 9 - Jun 15	Complete test suite (17 tests): fit/predict, save/restore, embeddings (mean + CLS), frozen backbone, long/short sequence edge cases. All passing. Midterm PR open.

PHASE 2: Benchmarks and Datasets (Weeks 4–6)

WEEK	DATES	DELIVERABLES
4	Jun 16 - Jun 22	Genomics dataset loader in dc.molnet. load_encode_tfbs() downloading and splitting ENCODE ChIP-seq. Scaffold split confirmed working.

5	Jun 23 - Jun 29	Benchmark run: NT v2-100m and v2-500m on ENCODE ChIP-seq (690 TF binding tasks), splice-site detection, promoter prediction. AUROC vs k-mer RF baseline.
6	Jun 30 - Jul 6	Benchmark write-up. Results table in docs. Model cards written. PR for dataset loader merged.

PHASE 3: Pre-training Scaffold (Weeks 7–9)

WEEK	DATES	DELIVERABLES
7	Jul 7 - Jul 13	pretrain() API design confirmed with mentors. MLM masking strategy (15% of 6-mer tokens) implemented.
8	Jul 14 - Jul 20	Full MLM training loop: forward pass with LM head, masked token loss, gradient accumulation. Loss decreasing on held-out sequences.
9	Jul 21 - Jul 27	Pre-training evaluation: fine-tune post-pre-training vs direct fine-tune on splice-site dataset. Report improvement. PR opened.

PHASE 4: Geneformer and Documentation (Weeks 10–12)

WEEK	DATES	DELIVERABLES
10	Jul 28 - Aug 3	GenformerModel wrapper: scRNA-seq tokenisation, fit(), predict(), evaluate() on Zheng 68k cell-type classification.
11	Aug 4- Aug 10	Cell-type benchmark. AUROC ≥ 0.90 on held-out cell types. Single-cell tutorial notebook.
12	Aug 11 - Aug 18	All PRs merged or in final review. Sphinx API documentation complete. Tutorial notebooks published. GSoC final report written and submitted.

10. Testing And Benchmarking Plan:

Unit Tests — All unit tests run on CPU without a GPU, using small randomly-generated DNA sequences and a small pre-downloaded checkpoint (v2-100m at <1 GB). Components tested:

- DNATokenizerFeaturizer: output shape, integer dtype, padding behaviour, truncation of long sequences, attention mask stacking
- KMerDNAFeaturizer: shape for k=3 and k=6, normalisation sums to 1.0, count mode, N-base handling

- NucleotideTransformerModel: classification fit/predict, regression fit/predict, predictions are finite, save/restore round-trip, frozen backbone training, pretrain() raises NotImplementedError
- GenformerModel: cell-type prediction shape, fit/predict smoke test

Integration Tests

- End-to-end pipeline: raw DNA strings → NumpyDataset → fit() → evaluate() on ENCODE sample
- Save/restore fidelity: predictions before and after restore() match within 1e-4
- get_embeddings() shape and finiteness: mean and CLS pooling, both backbone sizes

3. Benchmarking -

Benchmarking

Task	Dataset	Baseline	NT v2-100m	NT v2-500m
TF binding	ENCODE ChIP-seq (690 tasks)	k-mer RF AUROC	Target ≥ 0.93	Target ≥ 0.95
Splice site	Splice-AI	k-mer RF AUROC	Target ≥ 0.94	Target ≥ 0.96
Promoter	ENCODE	k-mer RF AUROC	Target ≥ 0.95	Target ≥ 0.97
Cell-type	Zheng 68k	SVM baseline	Target ≥ 0.90	—

11. Why am I the right candidate ?

1. Working prototype submitted before the proposal deadline. The prototype at https://github.com/arjitrawat15/Nucleotide-Transformer-DNA-Model contains the model file, featurizers, and 17 passing tests. This is not a plan about what I intend to build — it is working code that demonstrates I already understand DeepChem's model architecture, TorchModel API, loss functions, default_generator protocol, and test conventions.

2. Deep familiarity with the exact reference implementations. My proof-of-work includes detailed code review of chemberta.py, molformer.py, and hf_models.py. The prototype's __init__ signature, default_generator override, loss selection logic, and head architecture all mirror these reference implementations exactly — the contribution will feel native to the codebase, not bolted on.

3. Active experimental work on LLM fine-tuning in DeepChem. I am currently fine-tuning OLMo-1B-hf on ClinTox, Tox21, and HIV on an NVIDIA B200 MIG slice, using

DeepChem's scaffold splitter and dataset infrastructure. This gives me firsthand experience with every layer of the stack this project extends.

4. I have already solved real problems in this domain. The featurizer='Raw' crash in dc.molnet.load_clintox, the device-mismatch error between backbone and head, the head–LoRA learning rate conflict — I have diagnosed and fixed each of these in practice, not in theory. I know what breaks when you integrate large HuggingFace models into DeepChem.

THANK YOU FOR CONSIDERING THIS PROPOSAL

Arjit Rawat - arjitrawat3@gmail.com - github.com/arjitrawat15